

WHOLESALE BEAUTY HUB

WBH Skin

Technical Handover & Complete System Documentation

Evidence-based audit of the full application source code: product, frontend, backend, AI skin-condition analysis, database, security, privacy, and maintenance.

Application: WBH Skin — Wholesale Beauty Hub AI Skin Diagnostics (package name `wbh`, repository `dermaAI_web`)

Repository: https://github.com/OSAMOR19/dermaAI_web (branch `main`)

Audit date: 2 September 2026

Audit method: Full manual + automated inspection of the source tree (frontend, API routes, libraries, SQL schema files, environment configuration, static assets). Every material claim in this document cites a file path, and where useful a line number.

Prepared for: Client handover — readable by both non-technical stakeholders and engineers.

Note on naming: the commissioning brief referred to the product as “Wholesale Beats Hub”. The source code, PRD, and branding identify the application as **WBH Skin — “Wholesale Beauty Hub AI Skin Diagnostics”** (PRD.md line 2). This document uses the name found in the code. No secrets, API keys, or credential values are reproduced anywhere in this document.

Table of Contents

1	Executive Summary	23	Security Audit
2	Product Functionality	24	Privacy & User Data
3	User Journey	25	Error Handling
4	Frontend Documentation	26	Performance Audit
5	Complete Screen / Page Inventory	27	Logging & Monitoring
6	Component Architecture	28	Deployment & Infrastructure
7	Authentication & User Management	29	Project Structure
8	AI Skin-Condition Detection — Core Audit	30	Tech Stack Inventory
9	AI Analysis Pipeline	31	Business Logic
10	Image Input System	32	AI Cost & Usage Considerations
11	AI Prompt Audit	33	Limitations of the Current Application
12	AI Response Structure	34	What the Client Owns / Depends On
13	Skin Condition Classification	35	Maintenance Guide
14	AI Confidence & Accuracy	36	How to Add a New Skin Condition
15	Medical / Safety Considerations	37	How to Change the AI Model
16	Backend Architecture	38	Testing Audit
17	Complete API Inventory	39	Complete Dependency Map
18	Database Audit	40	Source-Code Evidence Convention
19	Data Model	41	Unknown / Unverified Information
20	Image Storage & File Management	42	Final Client-Friendly Explanation
21	Third-Party Services	43	Final Technical Summary
22	Environment Variables	44	Final Audit Checklist

1 Executive Summary

1.1 What is WBH Skin?

WBH Skin is a web application for **Wholesale Beauty Hub**, a UK skincare retailer. It lets a person photograph their face (or upload a photo), has an artificial-intelligence model examine the image for common skin concerns, and then turns the result into a personalised skincare report: detected conditions, a skin-health score, a morning and evening routine, and specific Wholesale Beauty Hub products to buy. Users can also book a consultation with the company's resident aesthetician, and staff have an admin dashboard to review scans, manage the product catalogue, and send curated product recommendations by email.

1.2 What problem does it solve?

Shoppers rarely know which skincare products suit their skin. WBH Skin replaces guesswork with an instant AI assessment and connects the result directly to the shop's catalogue — giving customers scientific-sounding, personalised reasoning for a purchase, and giving the business a lead-generation and conversion tool (the PRD states the objectives as increasing e-commerce conversion and streamlining lead acquisition — PRD.md §2).

1.3 Who is it for?

- **Consumers** (B2C shoppers, with copy specifically addressing melanated/darker skin tones) who want to understand their skin and get product guidance.
- **The in-house aesthetician** (Evelyn Badaiki), who receives consultation bookings by email.
- **WBH staff/administrators**, who use the admin portal for user management, scan review, catalogue management, event-lead follow-up, and sending recommendations.

1.4 What does the application allow users to do?

- Create an account (email/password or Google), verify email, reset passwords.
- Run an **AI skin scan** from the live front camera or by uploading a photo, gated behind a biometric-consent checkbox and real-time image-quality checks.
- View an **analysis report**: skin score, up to 12 recognised skin conditions with confidence, severity, observations and suggested active ingredients.
- Confirm which detected conditions they recognise, then receive **product recommendations** (up to 6 products) with a morning/evening routine and a list of things to avoid.
- Keep a **scan history** (images + results), re-open or delete past scans.
- **Book a consultation** with the aesthetician (date/time/notes) and download an SVG/PNG booking pass; both parties get confirmation emails.
- Register for events via a public lead-capture form (no login needed).
- Manage a profile (name, age range, country, skin type, avatar) and toggle light/dark theme.
- Jump out to the Wholesale Beauty Hub WooCommerce shop for purchases (no in-app checkout exists).

1.5 What role does AI play?

AI is the core diagnostic engine. The app sends the user's face image to **Google Gemini 2.5 Flash** (falling back to Gemini 2.0 Flash) with a fixed instruction prompt. Gemini validates the image (face present, single face, adequate quality) and classifies it against a strict list of **12 skin conditions**, returning structured JSON with per-condition confidence, severity, observations, explanation and suggested active ingredients. A second, smaller Gemini call writes a friendly two-sentence summary for the results page. Product matching itself is *not* AI — it is a deterministic, rule-based recommendation engine that maps detected concerns to tagged products in the database (src/lib/recommendation-engine.ts).

1.6 High-level application workflow

```
User (browser, mobile-first PWA)
  ↓ consents to biometric processing
Camera capture (640×480 JPEG) or photo upload (≤20 MB JPEG/PNG)
  ↓ client-side canvas checks: lighting 30–230 luminance, sharpness > 7.0
POST /api/analyze (Next.js serverless route – image → base64)
  ↓
Google Gemini 2.5 Flash (retry ×3 → fallback Gemini 2.0 Flash)
  ↓ strict-JSON prompt: validation + 12-condition classification
Structured analysis JSON (conditions, confidence, severity, skin type, disclaimer)
  ↓
Results page (/analysis): score, condition cards, disclaimer
  ↓ user ticks conditions they recognise
POST /api/recommend → Supabase products table → rule-based matching
  + Gemini text call for a 2-sentence friendly summary
  ↓
Personalised routine + 6 product cards linking to wholesalebeautyhub.co.uk
  | in parallel: image uploaded to private Supabase Storage,
  | scan row saved to Postgres (scans table) for history
Optional: consultation booking → Supabase consultations + Resend emails
```

Headline caveats for the client — detailed later in this document: (1) several admin API endpoints and the event-registration lead list are currently accessible *without any authentication*, exposing personal data and allowing catalogue tampering (Section 23); (2) product **prices shown to users are fabricated** by a hashing function, not real shop prices (Section 31); (3) there is **no validated clinical accuracy figure** anywhere in the code — the AI output is an estimation, not a diagnosis (Section 14); (4) there are **no automated tests and no CI/CD** (Section 38); (5) the repository README is stale boilerplate from a different project, and three database columns the admin system depends on exist only in the live database, not in the SQL files (Section 18).

2 Product Functionality

Every feature found in the source, with implementation status. Status legend: **IMPLEMENTED** **PARTIAL** **MOCK / UI-ONLY**

LEGACY / UNUSED

Feature	What it does, and how	Status
Marketing landing page	Public page at <code>/</code> : hero with animated mock scanner, product carousel (9 hardcoded products), testimonials carousel (10 hardcoded), trust marquee, footer, floating WhatsApp button. All shop links go to <code>wholesalebeautyhub.co.uk</code> . <code>src/app/page.tsx</code> , <code>src/landing/components/*</code>	IMPLEMENTED
Sign-up	Email/password with first/last name, age range, country; terms modal with full ToS/Privacy copy; email verification via Supabase; Postgres trigger auto-creates the profile row. <code>src/app/(auth)/signup/page.tsx</code> ; <code>supabase/schema.sql:40-65</code>	IMPLEMENTED
Login / OAuth	Email/password + Google OAuth (Apple is typed in code but has no button). “Remember me” checkbox is inert. <code>src/app/(auth)/login/page.tsx</code>	IMPLEMENTED
Password reset	Request email → Supabase reset link → <code>/reset-password</code> sets the new password then signs the user out. <code>(auth)/forgot-password</code> , <code>(auth)/reset-password</code>	IMPLEMENTED
Biometric consent gate	Checkbox + policy modal must be accepted before camera or upload unlocks. Consent is UI-state only — never recorded in the database. <code>src/app/(main)/scan/page.tsx:79, 382-424, 689-747</code>	PARTIAL
Live camera scan	Front camera 640×480; a 200 ms loop runs canvas lighting/sharpness checks and drives 4 indicator lights (2 of the 4 — “position” and “angle” — are simulated from the other two); shutter unlocks when all pass; capture exports JPEG at 85% quality; WebAudio shutter/beep sounds. <code>scan/page.tsx:110-191</code> ; <code>src/lib/imageValidation.ts</code>	IMPLEMENTED
Photo upload scan	File picker (JPEG/PNG/HEIC label), 20 MB cap, same canvas quality gate. Note: HEIC will fail decoding in most browsers and shows “Corrupted image file”. <code>scan/page.tsx:214-232, 466</code>	IMPLEMENTED
AI analysis	Image(s) → <code>/api/analyze</code> → Gemini 2.5/2.0 Flash with a strict 12-condition JSON prompt; retries on 429/503 with exponential backoff; result parsed and returned. <code>src/app/api/analyze/route.ts</code>	IMPLEMENTED
Analysis report	Skin score circle, condition cards with confidence bars, severity, observations, active ingredients, expandable details, image-quality warnings, share (navigator.share/clipboard). “Scan Diagnostics” rows (lighting/sharpness/face) are hardcoded “Optimal/Clear/Single Subject”; the “Report” download button has no handler. <code>src/app/(main)/analysis/page.tsx:399-412, 717</code>	PARTIAL
Condition confirmation step	User must tick which detected conditions they recognise; only the confirmed subset is sent for recommendations. <code>analysis/page.tsx:176-210</code>	IMPLEMENTED
Product recommendations	<code>/api/recommend</code> queries active products, runs the deterministic tag-matching engine, builds morning/evening routine, “avoid” list, and a Gemini-written 2-sentence summary; returns up to 6 products. Displayed prices are generated from a name-hash, not real prices. <code>src/app/api/recommend/route.ts</code> ; <code>src/lib/recommendation-engine.ts</code>	PARTIAL
Scan history	Scans saved (score, JSONB analysis, storage paths); history list with re-open, single delete, clear-all; images served by 1-hour signed URLs. <code>/api/scans</code> ; <code>profile/[slug] ScanHistory</code>	IMPLEMENTED
Consultation booking	One hardcoded specialist (Evelyn Badaiki, £50); 5 fixed time slots; date validation; dedupe check; row saved to consultations; Resend emails to booker and specialist; SVG booking pass with PNG download. No payment is taken although the email says “Fee Status: Paid”. <code>(main)/booking/*</code> ; <code>/api/booking</code>	PARTIAL
Event registration	Public form (name, email, phone, age, location, skin-concern chips) → <code>event_registrations</code> table via service-role key → confirmation email to registrant + notification to admin. <code>(auth)/event-registration</code> ; <code>/api/event-registration</code>	IMPLEMENTED
Profile management	Edit name/age/country/skin type; avatar resized client-side to 150×150 and stored as base64 text in <code>profiles.avatar_url</code> (the avatars storage bucket exists but is unused). <code>profile/[slug]/page.tsx:46-250</code>	IMPLEMENTED
Payment methods screen	“No saved cards” notice; billing-address form concatenates city/postcode into <code>profiles.location</code> and never reads them back. No payment processor exists anywhere. <code>profile/[slug]:254-373</code>	MOCK / UI-ONLY
Notification settings	Six toggles — local state only, nothing persisted or acted on. <code>profile/[slug]:376-417</code>	MOCK / UI-ONLY

Privacy & security screen	2FA / biometric / data-sharing toggles, Change Password, Delete Account — all inert. There is no account-deletion capability anywhere . <code>profile/[slug]:488-558</code>	MOCK / UI-ONLY
App settings	Dark mode works (localStorage + data-theme); Reduced Motion, Language, and “Clear Cache (24 MB)” are inert. <code>profile/[slug]:420-485</code>	PARTIAL
Video call screen	<code>/call</code> renders a static mock (image + four dead buttons). Not linked from navigation. <code>(main)/call/page.tsx</code>	MOCK / UI-ONLY
Shop	<code>/shop</code> is an interstitial that auto-opens the external WooCommerce store; the bottom-nav Shop tab links straight out. <code>(main)/shop/page.tsx</code> ; <code>Navbar.tsx</code>	IMPLEMENTED
PWA	manifest.json (standalone, pink theme) + service worker that precaches key routes and caches every same-origin GET, network-first. <code>public/manifest.json</code> ; <code>public/sw.js</code> ; <code>layout.tsx:57-63</code>	IMPLEMENTED
Cookie consent banner	Accept/Decline stored in localStorage; declining changes no behaviour (there are no analytics cookies to block). <code>src/components/CookieConsent.tsx</code>	PARTIAL
Admin: PIN gate + overview	PIN checked by <code>/api/admin/verify-pin</code> (hardcoded fallback PIN in source), flag kept in sessionStorage; overview shows stat cards, scans-per-day bar chart, top-conditions donut (Recharts), recent activity. <code>(admin)/admin/page.tsx</code> ; <code>AdminOverviewClient.tsx</code>	PARTIAL
Admin: user management	Paginated searchable user list; user detail with scans (signed images), consultations, plan (free/pro) and role (user/admin) editing. <code>(admin)/admin/users/*</code> ; <code>/api/admin/users*</code>	IMPLEMENTED
Admin: scan browser + activity	Filterable scan lists with image lightbox, severity badges, confidence bars. <code>(admin)/admin/scans</code> , <code>/admin/activity</code>	IMPLEMENTED
Admin: product catalogue	List/dashboard views, filters, product detail, recommendation counts. The PATCH update API applies the raw request body with no field allowlist and no auth. <code>(admin)/admin/products/*</code> ; <code>/api/admin/products*</code>	PARTIAL
Admin: recommendations	Generate from a user's scans, hand-pick products, add notes, finalize → emails the user + info@ + sales@. <code>admin/users/[id]/UserDetailClient.tsx</code> ; <code>/api/admin/recommendations*</code>	IMPLEMENTED
Admin: event registrations	Lead list with per-registrant product recommender (auto-match by concerns), copy-to-clipboard, Save & Send email, CSV export. <code>(admin)/admin/registrations/page.tsx</code>	IMPLEMENTED
Admin: settings	Feature-flag toggles and platform info — entirely cosmetic; “Save” only flashes a message. <code>(admin)/admin/settings/SettingsClient.tsx</code>	MOCK / UI-ONLY
Calendly integration	Four env vars provisioned and one query-param comment; zero functional code. <code>.env.local</code> names; <code>booking/success/page.tsx:21</code>	LEGACY / UNUSED
React Native app	<code>_rn/</code> is an Expo scaffold not wired to anything; the root README still documents it. <code>_rn/*</code> ; <code>README.md</code>	LEGACY / UNUSED
Static product data	<code>src/data/products.json</code> (12 conditions → products) is imported nowhere; live recommendations read the database. <code>src/data/products.json</code>	LEGACY / UNUSED

3 User Journey

3.1 Primary journey — first scan to recommendation (verified against source)

```
Open https://<site>/ (public landing, marketing content)
  ↓ "Start AI Scan" / Login
/login or /signup ————— email+password or Google OAuth
  ↓ (signup: "Check Your Email" → click link → /auth/callback → session)
/dashboard ————— greeting, empty-state "Ready for your first scan?"
  | CompleteProfileModal may ask age range + country
  ↓ "Start Scan"
/scan ————— tick Biometric Consent (required)
  ↓ choose: Live Camera | Upload Photo
  camera: 4 quality indicators must pass → shutter → review
  upload: ≤20 MB JPEG/PNG → canvas lighting/sharpness gate → review
  ↓ "Analyze"
POST /api/analyze (120 s timeout, rotating status text)
  | validation_error: no_face / multiple_faces / low_quality → friendly retry message
  | 503 "AI is experiencing very high demand" → retry message
  | success → analysis stored in sessionStorage, image uploaded to
  | Supabase Storage, scan row saved via POST /api/scans
  ↓ (1.8 s success animation)
/analysis ————— score, condition cards, medical disclaimer
  ↓ user ticks recognised conditions → "Confirm N Conditions"
POST /api/recommend ————— routine + up to 6 product cards + avoid list
  ↓ options
  "Book Consultation" → /booking → /booking/evelyn → POST /api/booking
    → /booking/success (downloadable booking pass; 2 confirmation emails)
  product card link → wholesalebeautyhub.co.uk product page (external)
  Share → navigator.share / clipboard
```

3.2 Alternative and error flows

- **Not logged in:** middleware redirects every non-public page to `/login` (the intended destination is not preserved).
`src/lib/supabase/middleware.ts:49-53`
- **Anonymous scan:** `/scan` is behind login, so this flow cannot occur through the UI; however the API itself would accept it (Section 23).
- **Image fails validation client-side:** inline message ("too dark or overly exposed...", "too blurry...") and the user retakes.
`src/lib/imageValidation.ts:100-106`
- **Gemini rejects the image:** `validation_error` mapped to human text: "We couldn't detect a face...", "Multiple faces detected...", "Image quality too low...". `scan/page.tsx:284-296`
- **AI overloaded:** HTTP 503 after retries → "Our AI is experiencing very high demand right now." `analyze/route.ts:177-182`
- **120-second client timeout:** `AbortController` cancels and shows an error state. `scan/page.tsx:265-273`
- **Scan save fails:** silently ignored — the user still sees "Scan complete", but the scan is missing from history. `scan/page.tsx:331-334`
- **Returning user:** `/analysis` falls back from `sessionStorage` to the latest scan in the database; "no data" empty state links back to `/scan`. `analysis/page.tsx:107-140, 258-276`
- **Public lead:** `/event-registration` works without an account and ends at a success card.

4 Frontend Documentation

Aspect	Implementation
Framework	Next.js 15.3 (App Router, Turbopack in dev), React 19, TypeScript 5.9. Package name <code>wbh</code> .
Architecture	Three App Router route groups: <code>(main)</code> — the authenticated app with a bottom tab bar, <code>(auth)</code> — auth pages + the public event form, <code>(admin)</code> — a self-contained admin portal that renders its own <code><html><body></code> (a second root layout, without ThemeProvider or CookieConsent). Almost every page is a client component; the landing page and <code>/call</code> are server components.
Routing	File-based. Dynamic segments: <code>/booking/[id]</code> , <code>/profile/[slug]</code> (a client-side sub-router mapping 7 sub-pages inside one 874-line file), <code>/admin/users/[id]</code> , <code>/admin/products/[id]</code> .
State management	React only — two contexts (<code>AuthProvider</code> , <code>ThemeProvider</code>); everything else is local <code>useState</code> / <code>useEffect</code> . No Redux/Zustand/React Query/SWR.
Data fetching	Plain <code>fetch()</code> to internal <code>/api/*</code> routes, plus direct browser Supabase queries in the profile editor and AdminSidebar. Hand-rolled sessionStorage caches: scan list 60 s, profile 5 min.
Forms & validation	All controlled inputs, all validation manual (if-chains setting error strings). No form or schema library. Password rule: minimum 6 characters, client-side only.
Styling	No Tailwind. Three layers: <code>globals.css</code> (1,661 lines, CSS custom properties, <code>[data-theme="dark"]</code> block), <code>admin.css</code> (1,214 lines), and very heavy inline <code>style={{...}}</code> plus scoped <code><style></code> blocks. Brand pink <code>#e84c88</code> (an older pink <code>#fc65d1</code> persists in places). Dark mode only affects class-styled surfaces — inline-styled areas and the whole admin stay light.
Loading / error / empty states	Consistent: shimmer skeletons on the dashboard, spinners in forms and admin, empty-state cards, inline error panels; <code>alert()</code> used in 3 places.
Auth in the client	<code>AuthProvider</code> (<code>getSession</code> + <code>onAuthStateChange</code>) is mounted only in the <code>(main)</code> layout — auth pages and admin use the Supabase client directly.
Image handling	Canvas-based capture/resize/quality checks (Section 10). <code>next.config.ts</code> sets <code>images.unoptimized: true</code> (no Next.js image optimisation).
Browser storage	localStorage: <code>wbh_theme</code> , <code>wbh_cookie_consent</code> . sessionStorage: <code>wbh_analysis</code> , <code>wbh_scan_image</code> (full base64 face image), <code>wbh_scan_time</code> , <code>wbh_scans_cache(ts)</code> , <code>wbh_profile_cache(ts)</code> , <code>wbh_profile_dismissed</code> , <code>wbh_admin_auth</code> .
PWA / caching	<code>public/sw.js</code> : cache <code>wbh-skin-v1</code> , precaches <code>/</code> , <code>/login</code> , <code>/dashboard</code> , <code>/scan</code> , <code>/analysis</code> ; network-first, caches every same-origin GET (including authenticated pages — a shared-device risk). Registered by an inline script in the root layout.

5 Complete Screen / Page Inventory

Screen	Route	Purpose	Role	Backend / AI dependency
Landing	<code>/</code>	Marketing page: hero, product carousel, testimonials, footer, WhatsApp FAB	Public	None
Login	<code>/login</code>	Email/password + Google OAuth sign-in; friendly error mapping	Public	Supabase Auth
Sign-up	<code>/signup</code>	Registration with age range, country, terms modal; email-verify screen	Public	Supabase Auth + profile trigger
Forgot / Reset password	<code>/forgot-password</code> , <code>/reset-password</code>	Reset request and completion (signs out after change)	Public / emailed link	Supabase Auth
Event registration	<code>/event-registration</code>	Public lead-capture form with concern chips	Public	POST <code>/api/event-registration</code> → DB + 2 Resend emails

OAuth/email callback	<code>/auth/callback</code>	Exchanges the code for a session, redirects	Public	Supabase Auth
Dashboard	<code>/dashboard</code>	Greeting, latest score, detected areas, recent analysis card, profile-completion modal	User	GET <code>/api/scans</code> , GET/PATCH <code>/api/profile</code>
Scan	<code>/scan</code>	Consent gate → camera/upload → quality gates → analysis → save	User	POST <code>/api/analyze</code> (Gemini), Storage upload, POST <code>/api/scans</code>
Analysis	<code>/analysis</code>	Results report, condition confirmation, recommendations, routine, share	User	GET <code>/api/scans</code> , POST <code>/api/recommend</code> (DB + Gemini)
Booking hub / detail / success	<code>/booking</code> , <code>/booking/[id]</code> , <code>/booking/success</code>	Specialist card → date/slot/notes form → ticket with PNG download	User	POST/GET <code>/api/booking</code> → DB + 2 Resend emails
Shop	<code>/shop</code>	Interstitial that opens <code>wholesalebeautyhub.co.uk</code>	User	None
Call	<code>/call</code>	Static video-call mockup (non-functional)	User	None
Profile hub	<code>/profile</code>	Menu of 7 sub-pages + sign out	User	—
Profile sub-pages (7)	<code>/profile/[slug]</code>	edit-profile, payment-methods, notifications, app-settings, privacy-security, scan-history, help-center	User	Supabase direct + <code>/api/scans</code> (several sub-pages are UI-only)
Admin overview	<code>/admin</code>	PIN gate → stats, charts (Recharts), recent activity	Logged-in user + client-side PIN	<code>/api/admin/verify-pin</code> , <code>/api/admin/stats</code>
Admin users / user detail	<code>/admin/users</code> , <code>/admin/users/[id]</code>	User list/search; detail with scans, consultations, plan/role editing, recommendation builder	Admin (API-enforced)	<code>/api/admin/users*</code> , <code>/api/admin/recommendations*</code> , <code>/api/admin/products</code>
Admin scans / activity	<code>/admin/scans</code> , <code>/admin/activity</code>	Scan browsers with filters, images, severity	Admin (API-enforced)	<code>/api/admin/scans</code>
Admin products / detail	<code>/admin/products</code> , <code>/admin/products/[id]</code>	Catalogue list/dashboard/detail	API currently unauthenticated	<code>/api/admin/products*</code>
Admin registrations	<code>/admin/registrations</code>	Lead list, per-lead product recommender, send email, CSV export	API currently unauthenticated	<code>/api/event-registration</code> (GET), <code>/api/admin/registration-recommendations</code>
Admin settings	<code>/admin/settings</code>	Cosmetic feature flags + platform info	—	None (local state)

31 distinct screens: 16 user/public route pages (plus 7 profile sub-screens inside `/profile/[slug]`) and 9 admin pages. Middleware protects all non-public pages with “any logged-in user”; role separation happens (inconsistently) at the API layer — see Section 23.

6 Component Architecture

Component	Path	Purpose & notable logic
AuthProvider	<code>src/components/AuthProvider.tsx</code>	Context {user, session, loading, signOut}; subscribes to Supabase auth state; signOut hard-navigates to <code>/login</code> . Mounted only in (main) layout.
ThemeProvider	<code>src/components/ThemeProvider.tsx</code>	light/dark via <code>data-theme</code> + <code>localStorage</code> <code>wbh_theme</code> ; mounted at root but not in the admin layout.
Navbar	<code>src/components/Navbar.tsx</code>	5-tab bottom bar: Home, Booking, Scan, Shop (external, new tab), Profile; active state from pathname.

CompleteProfileModal	src/components/CompleteProfileModal.tsx	Progressive profiling: if age_range/country missing → 2-step wizard; 5-min profile cache; session-scoped dismissal.
CookieConsent	src/components/CookieConsent.tsx	Delayed banner; stores accepted/declined; decline is behaviourally inert.
Landing suite (7)	src/landing/components/*	LandingNavbar (scroll-morphing pill + mobile overlay), LandingHero (mock scanner animation), LandingWhyChooseUs (marquee), LandingServices (9 hardcoded products carousel), LandingImageGallery (10 testimonials autoplay), LandingFooter, WhatsAppButton (placeholder number 447000000000).
AdminSidebar / AdminTopbar	src/app/(admin)/components/*	Sidebar fetches the user's profile role client-side, computes isSuperAdmin (hardcoded email or role) and redirects non-admins — decorative, not a security control.
Page-embedded components	various	TermsModal (signup), SubpageHeader/ToggleSwitch (profile), skeletons (dashboard), BookingSuccessContent (SVG ticket + PNG export), UserDetailClient (939 lines — the admin recommendation builder), ProductsClient, ActivityClient, SettingsClient.
Business-logic libraries	src/lib/*	recommendation-engine.ts (deterministic product matching), imageValidation.ts (canvas quality gates), supabase/{client,server,middleware,admin,storage}.ts (client factories + storage helpers).

7 Authentication & User Management

Provider: Supabase Auth (email/password + Google OAuth; Apple is referenced in types but has no UI). Sessions are cookie-based via `@supabase/ssr`; the middleware refreshes tokens on every page navigation and calls `auth.getUser()`, which verifies the JWT server-side rather than trusting the cookie.

- **Registration** — `signUp` with metadata `{first_name, last_name, age_range, country}`; a SECURITY DEFINER Postgres trigger (`handle_new_user`, `supabase/schema.sql:40-65`) copies the metadata into `public.profiles`. Email verification uses the Supabase-hosted templates in `emails/confirmation.html` / `reset-password.html` (pasted into the Supabase dashboard — not sent by app code).
- **Login** — `signInWithPassword`; Google via `signInWithOAuth` redirecting to `/auth/callback`, which calls `exchangeCodeForSession` and redirects to `?next` (default `/dashboard`). The `next` parameter is not validated (origin-prefixed, so host redirection is blocked, but arbitrary internal paths are possible).
- **Logout** — `signOut()` then a hard navigation to `/login`.
- **Password reset** — `resetPasswordForEmail` → emailed link → `updateUser({password})` → forced sign-out (good practice). Minimum length 6, client-side only.
- **Protected routes** — middleware treats `/`, `/login`, `/signup`, `/forgot-password`, `/reset-password`, `/auth/*`, `/event-registration` as public; everything else requires any session. **API routes are excluded from the middleware entirely** (`src/middleware.ts:18`) — each handler must protect itself.
- **Roles** — `profiles.role` ∈ `{user, admin, superadmin}` (column exists only in the live DB, not in the SQL files) plus a hardcoded superadmin email (`info@wbhskin.com`) repeated in five files. Roles are granted via `supabase/promote_evelyn.sql` or the admin user-detail screen.
- **Plans** — `profiles.plan` ∈ `{free, pro}` is stored and editable by admins but **nothing in the application changes behaviour based on it**; there is no billing.
- **Account deletion** — **NOT IMPLEMENTED**. The “Delete Account” button is inert; no API or UI path deletes a user. (Database cascades are in place if a user were deleted directly in Supabase.)

8 AI Skin-Condition Detection — Core Technical Audit

8.1 AI provider and models

Google Gemini, called through the raw Generative Language REST API (`generativelanguage.googleapis.com/v1beta/models/<model>:generateContent`). No SDK, no Vertex AI, no OpenAI/Anthropic/custom ML model, and no on-device model exist anywhere in the repository.

Use	Model	Where
Vision analysis of skin images	<code>gemini-2.5-flash</code> , automatic fallback to <code>gemini-2.0-flash</code>	<code>src/app/api/analyze/route.ts:58-61</code>
Two-sentence friendly result summary (text-only)	<code>gemini-2.5-flash</code>	<code>src/app/api/recommend/route.ts:6, 239-271</code>

Generation settings for the vision call: `temperature 0.2` , `maxOutputTokens 8192` , `responseMimeType "application/json"` (`analyze/route.ts:71-78`). For the summary: `temperature 0.4` , `maxOutputTokens 200` . No `safetySettings` and no structured-output `responseSchema` are supplied — JSON shape is requested by prose only. The API key is passed in the **URL query string** rather than a header (a logging-exposure hygiene issue, Section 23).

9 AI Analysis Pipeline

1	Image input	camera JPEG (0.85 q, 640×480) or upload ≤20 MB (<code>scan/page.tsx</code>)
2	Client validation	canvas luminance 30–230, edge-sharpness > 7.0 (<code>lib/imageValidation.ts</code>)
3	Consent gate	biometric checkbox required before either input mode
4	Transport	FormData "file0" → POST <code>/api/analyze</code> (120 s <code>AbortController</code>)
5	Server encoding	File → ArrayBuffer → base64 → Gemini <code>inline_data</code> part (NO server-side size/MIME/count validation – see Security)
6	AI request	<code>parts = [DERMA_PROMPT text, image parts]</code> ; temp 0.2; JSON mime
7	Retry logic	per model: 3 attempts on 429/503 with 2 s/4 s backoff; then next model; exhaustion → "AI_OVERLOADED" → HTTP 503
8	Raw response	<code>candidates[0].content.parts[0].text</code>
9	Parsing	JSON.parse; fallback: extract from ``json fences`` via regex
10	Validation	NONE – the parsed object is returned verbatim (<code>route.ts:216</code>)
11	Client mapping	<code>validation_error</code> → friendly message; else analysis rendered
12	Persistence	<code>sessionStorage (wbh_analysis)</code> + Supabase Storage upload + POST <code>/api/scans</code> (score = 100 – average condition confidence)
13	Recommendation	confirmed conditions → POST <code>/api/recommend</code> → <code>products table</code> → deterministic engine → routine + products + second Gemini call for the 2-sentence summary (graceful fallback)

Two engineering gaps worth fixing: (a) the AI response is never validated server-side — a malformed or adversarial model output flows straight into the database and the UI (e.g. a confidence of 150 renders an overflowing bar, `analysis/page.tsx:589`); (b) a non-retryable error (e.g. HTTP 400) on `gemini-2.5-flash` aborts immediately and never tries the `2.0-flash` fallback (`analyze/route.ts:98-100`).

10 Image Input System

Property	Behaviour (verified)
Input modes	Live front camera (<code>getUserMedia</code> , <code>facingMode "user"</code> , ideal 640×480) or single file upload. Drag-and-drop: no. Multiple images: the UI caps at <code>MAX_IMAGES = 1</code> (<code>scan/page.tsx:13</code>); the API accepts up to 5 fields (<code>file</code> , <code>file0..file3</code>).

Accepted formats	Picker: <code>image/jpeg, image/png, image/heic</code> . HEIC is accepted by the picker but cannot be decoded by the canvas validator in most browsers → fails as “Corrupted image file”. Server: no MIME restriction at all.
Max size	20 MB, enforced client-side only (scan/page.tsx:217). No server cap.
Compression / resizing	Camera path: canvas export JPEG at 85% quality at capture resolution. Upload path: no resize or re-compression — a 19 MB photo goes to Gemini at full size (base64 inflates it ~33%).
Quality validation	Canvas sampling every 4th pixel: perceived luminance must be 30–230, mean horizontal edge delta > 7.0 (imageValidation.ts:51-53). Camera mode shows 4 live indicators — “Position” and “Angle” are derived from the other two, i.e. simulated (scan/page.tsx:160-162).
EXIF / orientation	No EXIF handling anywhere; orientation relies on browser decoding.
Encoding chain	dataURL → Blob (client) → multipart → ArrayBuffer → base64 <code>inline_data</code> (server).
Storage	After a successful analysis and only when logged in: upload to the private Supabase Storage bucket <code>scans</code> at <code>{userId}/{scanId}/{index}.jpg png</code> (lib/supabase/storage.ts). The full base64 image is also cached in sessionStorage (<code>wbh_scan_image</code>).

11 AI Prompt Audit

Exactly two prompts exist in the application. Neither uses a system-instruction field — each is a single user-turn text part. Reproduced verbatim (no secrets are contained in them).

11.1 DERMA_PROMPT — vision analysis (src/app/api/analyze/route.ts:6-56)

```
You are an AI dermatology analysis assistant.

The user has uploaded one or more skin images.
First, perform validation on the image(s):
1. Check if there is a human face/skin area in the image. If there are NO faces or clear skin areas, set "validation_error" to "no_face_detected".
2. Check if there are MULTIPLE human faces present in the image(s). If you detect more than one face, set "validation_error" to "multiple_faces_detected".
3. Check if the image quality is too low (e.g., extremely blurry, out of focus, extremely dark, or not a skin image). If so, set "validation_error" to "low_quality".

If "validation_error" is set, you do NOT need to perform skin condition analysis. Just return the JSON with the "validation_error" field populated and "detected_conditions" as an empty array.

Otherwise, analyze the image(s) for signs of the following strict list of 12 skin conditions:

- Acne (Inflammatory)           - Acne (Comedonal)
- Hyperpigmentation             - Melasma
- Dark Spots / Sun Damage       - Rosacea
- Dehydration                   - Oily Skin / Excess Sebum
- Fine Lines / Wrinkles         - Dark Circles
- Enlarged Pores                - Uneven Skin Tone

Return the response in the following strict JSON format ONLY:
{
  "image_quality": "good" | "poor",
  "validation_error": null | "no_face_detected" | "multiple_faces_detected" | "low_quality",
  "images_analyzed": 1,
  "detected_conditions": [
    { "condition": "Must be an EXACT MATCH to one of the 12 allowed conditions above (e.g. 'Melasma' or 'Acne (Inflammatory)')",
      "confidence": 85,
      "observations": ["visible redness", "raised bumps"],
      "severity": "Moderate",
      "clinical_explanation": "A very brief 1-sentence explanation of this condition based on the observations, strictly avoiding the word 'clinical'.",
      "active_ingredients": ["Salicylic Acid", "Niacinamide"] } ],
```

```
"skin_type_estimate": "oily" | "dry" | "normal" | "combination",
"confidence_score": 85, // Overall analysis confidence/quality score (0-100) based on
                        // image clarity and quality
"disclaimer": "This AI analysis is for informational purposes only and does not
               constitute medical advice."
}
If no condition is detected, return an empty array for detected_conditions.
Do not return any explanation outside the JSON object.
```

Business logic embedded in this prompt: the image-validation gate (face present / single face / adequate quality), the closed 12-condition taxonomy, severity labelling, the requirement to suggest cosmetic active ingredients (which feeds product matching), and the built-in medical disclaimer. Weaknesses: severity values are never enumerated (downstream code assumes “Severe/Moderate/Mild”), the JSON example contains a `//` comment which invites invalid JSON, and no machine-enforced schema backs the format request.

11.2 Summary prompt (src/app/api/recommend/route.ts:241-252)

```
You are a friendly skincare assistant for people with melanated (dark) skin.

Given these skin concerns: ${primary_concerns}${", and minor: " secondary_concerns}
Skin type: ${skin_type_estimate || 'unknown'}

Write a SHORT, friendly, 2-sentence summary. Rules:
- Use simple English, no medical jargon
- Say "it looks like" or "your skin shows signs of" – NEVER say "you have" a disease
- Be encouraging and positive
- Must be exactly 2 sentences, nothing else
- Do NOT use markdown, bullet points, or formatting
- Return ONLY the 2 sentences, no JSON, no quotes
```

The interpolated values come from the client request body without an allowlist check — a caller can inject arbitrary text into this prompt (low impact: output is a 200-token plain-text summary, but it is user-visible).

12 AI Response Structure

Field	Type / values	Consumed by
image_quality	"good" "poor"	Quality warning banner on /analysis
validation_error	null no_face_detected multiple_faces_detected low_quality	Mapped to friendly retry messages (scan/page.tsx:284-296)
images_analyzed	number	Informational
detected_conditions[]	{condition, confidence 0-100, observations[], severity, clinical_explanation, active_ingredients[]}	Condition cards; concern extraction (≥30 confidence) for recommendations; stored in scans.analysis
skin_type_estimate	oily dry normal combination	Displayed; augments concerns (oily/combo → adds “Oily Skin / Excess Sebum”, dry → “Dehydration”)
confidence_score	0-100 (image-quality confidence)	Displayed as the headline score (fallback: locally computed)
disclaimer	string	Rendered if present

Fallbacks: unparseable AI text → markdown-fence extraction → 502 “AI returned invalid response format”; empty response → 502; overload → 503 with a friendly message; summary-call failure → deterministic template sentence (recommend/route.ts:273-275). **There is no server-side validation of the parsed object** — fields are trusted as returned.

13 Skin Condition Classification

Condition	Detection	Where the taxonomy lives	Result handling
-----------	-----------	--------------------------	-----------------

Acne (Inflammatory)	All 12 detected by the same Gemini prompt; no per-condition model logic	Prompt (analyze/route.ts:16-29); AVOID_MAP (recommend/route.ts:21-34); CONCERN_TAG_MAP (recommendation-engine.ts:38-84); legacy src/data/products.json	Condition card → user confirmation → tag expansion → product matching → routine + avoid list. Severity Severe/Moderate → “primary concern”, else secondary.
Acne (Comedonal)			
Hyperpigmentation			
Melasma			
Dark Spots / Sun Damage			
Rosacea			
Dehydration			
Oily Skin / Excess Sebum			
Fine Lines / Wrinkles			
Dark Circles			
Enlarged Pores			
Uneven Skin Tone			

Non-detection outcomes the system supports: “no condition detected” (empty array → “Healthy Skin”), “unable to analyze” (the three validation_error codes), and multiple simultaneous conditions (an array). There is no “Unknown/Uncertain” category; nothing stops the model returning an off-list condition string, since exact-match enforcement is prompt-only — an off-list string would render in the UI and simply match no products.

14 AI Confidence & Accuracy

- Confidence is **AI-self-reported**: per-condition 0–100 and an overall image-quality score, both generated by Gemini in the same response. There is no separate classifier, probability calibration, threshold tuning, or human review loop.
- The only threshold in the code is `confidence >= 30` for including a condition in recommendation concerns (recommendation-engine.ts:271).
- The stored “skin score” is `100 - average condition confidence` (scan/page.tsx:312-315) while the displayed headline score prefers the AI’s image-quality `confidence_score` — two different semantics that can disagree.

No validated clinical/model accuracy figure was identified in the application source code. Any accuracy, sensitivity, or specificity number quoted in marketing must be produced and validated separately — nothing in the repository substantiates one. The confidence numbers shown to users are the language model’s own uncalibrated estimates.

15 Medical / Safety Considerations

The product is positioned as **AI estimation / cosmetic classification**, not clinical diagnosis. Safety measures present in the code:

- Hardcoded disclaimer banner** on every analysis view: “...intended for informational purposes only and does not constitute medical advice, diagnosis, or treatment...” (analysis/page.tsx:330-338), plus the model-returned disclaimer field when present (analysis/page.tsx:706-710) and a disclaimer line in booking emails (booking/route.ts:128).
- Biometric consent gate** with a policy modal before any capture/upload (scan/page.tsx:382-424, 689-747). The policy states facial coordinates are processed in real time and raw biometric templates are not stored. Note: the *photographs themselves are stored* (Supabase Storage + sessionStorage), so this wording needs legal review.
- Cautious language rules** in the summary prompt (“NEVER say ‘you have’ a disease”).
- Human referral path**: the analysis page CTA routes users to a consultation with the in-house aesthetician (not a doctor/dermatologist).
- Image-quality gates** reduce garbage-in results (client canvas checks + model-side validation errors).

Gaps: no emergency or “see a doctor” warning for potentially serious presentations (the 12-condition list is cosmetic, and the model is not asked to flag anything outside it); false-negative risk is therefore structural — a malignant lesion would at best be forced into a cosmetic category or ignored; consent is not recorded; there is no age gate. The 12-condition taxonomy and referral-to-aesthetician design keep the product on the cosmetic side, but regulatory classification (e.g. UK medical-device rules) has not been assessed anywhere in the repo and requires professional review — see Section 41.

16 Backend Architecture

The backend is **serverless Next.js**: there is no separate server codebase. All server logic lives in App Router **route handlers** under `src/app/api/**/route.ts` (TypeScript, Node runtime). There are no “use server” actions, no background jobs, no queues, no GraphQL — plain REST-ish JSON endpoints. Persistence and auth are delegated to Supabase (Postgres + Auth + Storage); AI to Google Gemini; email to Resend.

```
Browser (React 19 client components, PWA)
|
|  cookies (Supabase session, refreshed by src/middleware.ts on page nav)
|
▼
Next.js route handlers  src/app/api/* — 18 files / 28 endpoints
├── Supabase clients (src/lib/supabase/)
│   ├── client.ts      browser, anon key, RLS enforced
│   ├── server.ts      per-request cookie-scoped, anon key, RLS enforced
│   └── admin.ts        SERVICE ROLE key – bypasses RLS (8 route files)
├── Google Gemini REST  (analyze: vision · recommend: text summary)
├── Resend REST         (8 send sites, inline HTML templates)
└── Supabase Storage    ('scans' private bucket, signed URLs 1 h;
                        'avatars' public bucket – provisioned but unused)
Postgres (Supabase):  profiles · scans · consultations · products · categories
                      · recommendations · recommendation_items · event_registrations
                      + trigger handle_new_user, RLS policies per table
```

Layering: there is no controller/service/repository separation — each route handler does auth (sometimes), querying, business logic, and email templating inline. The only shared business modules are `recommendation-engine.ts` and the Supabase client factories. The admin auth guard (`requireAdmin`) is copy-pasted into four files instead of shared. Caching: HTTP-level only (next.config.ts sets `no-store` for /api). Logging: `console.*` only.

17 Complete API Inventory

Every endpoint found by sweeping `src/app/api/**/route.ts`, `src/app/auth`, and every `fetch(` call site. Auth column = what the handler itself enforces (middleware never runs for /api). “Service role” = queries bypass row-level security.

Method & endpoint	Purpose	Auth	DB / external	Source
POST /api/analyze	AI skin analysis: multipart images → Gemini → analysis JSON	NONE	Gemini x1 (retries/fallback)	api/analyze/route.ts:116
POST /api/recommend	Analysis JSON → product recommendations, routine, Gemini summary	NONE	products (service role); Gemini x1	api/recommend/route.ts:125
POST / GET / DELETE /api/scans	Save scan; list last 20 with 1 h signed image URLs; delete one or ALL (empty body ⇒ all)	Session	scans, Storage	api/scans/route.ts
GET / PATCH /api/profile	Read/update own profile (field allowlist)	Session	profiles	api/profile/route.ts

POST / GET /api/booking	Create consultation (dedupe) + 2 Resend emails; list own bookings	Session	consultations; Resend x2	api/booking/route.ts
POST /api/event-registration	Public lead form → insert + 2 Resend emails	None (by design)	event_registrations (service role); Resend x2	api/event-registration/route.ts:18
GET /api/event-registration	List ALL registrations + linked recommendations	NONE	event_registrations, recommendations (service role)	...route.ts:287
GET /auth/callback	OAuth / email-link code exchange → redirect	n/a	Supabase Auth	app/auth/callback/route.ts
POST /api/admin/verify-pin	Admin PIN check (hardcoded fallback PIN; no rate limit)	n/a	—	api/admin/verify-pin/route.ts
GET /api/admin/stats	Dashboard stats, charts data, recent activity	admin/superadmin role ✓	profiles, scans, Storage	api/admin/stats/route.ts
GET /api/admin/scans	Paginated scan browser with filters	admin role ✓	scans, profiles, Storage	api/admin/scans/route.ts
GET /api/admin/users	Paginated user list + search (injectable .or() filter)	admin role ✓	profiles, scans; auth.admin.listUsers	api/admin/users/route.ts
GET / PATCH /api/admin/users/[id]	User detail (scans, consultations); set plan/role	admin role ✓	profiles, scans, consultations, Storage	api/admin/users/[id]/route.ts
GET /api/admin/products	Catalogue list + facets (brands, tags)	NONE	products, categories (service role)	api/admin/products/route.ts
GET / PATCH /api/admin/products/[id]	Product detail; update with raw body, no field allowlist	NONE	products (service role)	api/admin/products/[id]/route.ts:48
GET /api/admin/products/stats	Catalogue dashboard metrics	NONE	products, categories, recommendation_items (service role)	api/admin/products/stats/route.ts
GET / POST / PATCH /api/admin/recommendations	Read any user's recommendations; generate from scans; update status — "finalized" fires 3 Resend emails	NONE	recommendations, recommendation_items, scans, products, profiles (service role); Resend x2-3	api/admin/recommendations/route.ts
POST / DELETE /api/admin/recommendations/[id]/items	Add/remove products on a recommendation	NONE	recommendation_items (service role)	.../items/route.ts

POST /api/admin/registration-recommendations	Send curated products to an event lead; creates a pre-confirmed auth user if the email is unknown; emails user + info@ + sales@	NONE	profiles, recommendations, recommendation_items (service role); auth.admin.createUser; Resend x2	api/admin/registration-recommendations/route.ts:54
--	--	------	--	--

Totals: 18 route files, 28 method+path endpoints. Correctly guarded: scans, profile, booking, admin/stats, admin/scans, admin/users(+id). Unauthenticated: analyze, recommend, event-registration GET, verify-pin (by nature), and the entire products/recommendations/registration-recommendations admin family — see Section 23 for impact and fixes.

18 Database Audit

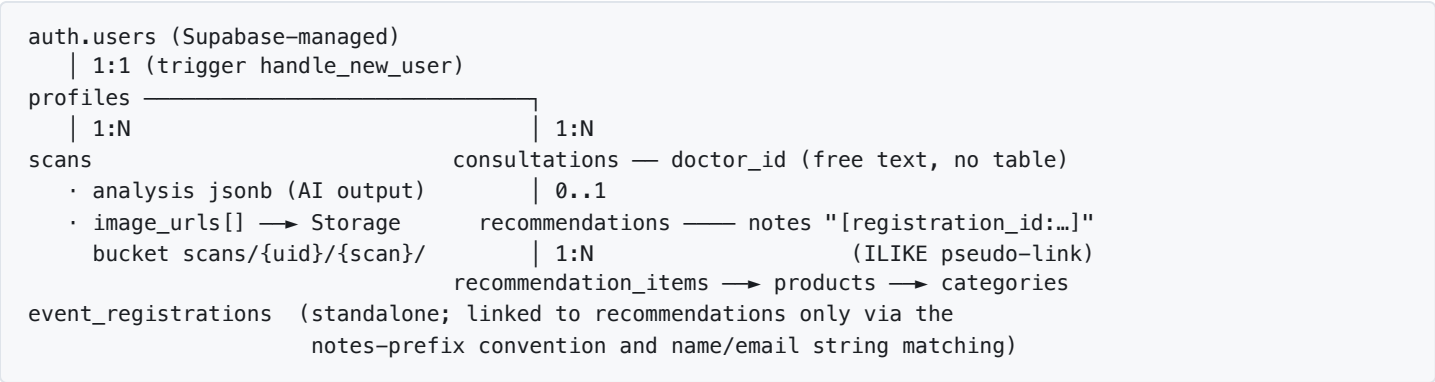
Technology: Supabase-hosted PostgreSQL. Schema is defined by hand-run SQL files (no migration tool). **Two divergent schema files define the core tables differently** (`supabase-setup.sql` older vs `supabase/schema.sql` newer/idempotent) and three columns used by the code (`profiles.role` , `profiles.plan` , `profiles.location`) exist in *no* SQL file — they were added manually in the dashboard. A fresh environment built from the repo alone would have a broken admin system.

Table	Purpose	Key columns	RLS (as written in SQL)
profiles	1:1 with auth.users; profile data	id uuid PK→auth.users (cascade); first/last name, email, phone, date_of_birth, age_range, country, avatar_url (stores base64 images), skin_type, created/updated_at; + undeclared role, plan, location	Owner-only SELECT/INSERT/UPDATE. UPDATE lacks WITH CHECK → a user could self-set role='admin' via the REST API (Critical, §23)
scans	AI analysis history	id uuid PK; user_id FK; image_urls text[] (storage paths); score int; analysis jsonb (raw Gemini output); recommendation_data jsonb; created_at	Owner-only, all four verbs ✓
consultations	Bookings	id uuid PK; user_id FK; doctor_id/doctor_name text (no doctors table); date & time stored as text ; notes; created_at; index (user_id, created_at desc)	Owner SELECT/INSERT/DELETE; no UPDATE policy (immutable)
categories	Product taxonomy (18 seeded)	id text PK ('CAT-01'); name; group_name; product_count (denormalised counter)	Public SELECT; service-role FOR ALL policy is written without WITH CHECK (defect repeated on all 4 product tables)
products	Catalogue (~310 rows after migration v2)	id text PK ('PROD-...'); category_id FK; category_name (denormalised); product_name; brand; sub_type; finish; best_for; confidence_level (High/Medium/Low); skin_concern_tags text[] (GIN indexed — drives matching); description; image_url(_2,_3); product_url; status (default 'active')	SELECT USING(true) — inactive products are publicly readable too , despite the policy name
recommendations	A curated product set for a user	id uuid PK; consultation_id FK nullable; user_id FK; skin_concerns text[]; status draft/sent/finalized (no CHECK); notes text — overloaded as a pseudo-FK (" [registration_id:...]" matched by ILIKE); created/updated_at (updated manually, no trigger)	Owner SELECT + service-role ALL

recommendation_items	Join: recommendation ↔ product	id uuid PK; recommendation_id FK cascade; product_id FK cascade; match_reason; concern_match[]; confidence_score int; added_by ('system'/'admin'); sort_order. No UNIQUE(recommendation_id, product_id) — dedupe is an app-level read-then-write race	SELECT via EXISTS on parent ownership ✓ + service-role ALL
event_registrations	Public event leads	id uuid PK; full_name, email NOT NULL; phone, age_range, location, postcode, address (last two never written by the API); skin_concerns[]; other_concern; created_at	CRITICAL — "Allow service role select" is written as USING (true) with no role restriction: anyone holding the public anon key can SELECT the whole lead table (supabase/event_registrations.sql:27-28)

Trigger/function: `handle_new_user()` SECURITY DEFINER, AFTER INSERT ON `auth.users` → upserts the profile from signup metadata. **Storage buckets:** `scans` (private) and `avatars` (public, unused) — Section 20. **Data retention:** nothing expires automatically; users can delete their own scans; no other deletion paths exist. **Migration tooling:** `supabase/migrate_all_products.ts.js` (manual Node script parsing a TSV product list, upserting in chunks of 50, recomputing category counts).

19 Data Model



Notable modelling debts: consultation date/time as text; the notes-prefix pseudo-foreign-key for registration recommendations (string matching can cross-link people who share a name — `api/event-registration/route.ts:314-320`); denormalised `category_name` and `product_count` kept in sync manually.

20 Image Storage & File Management

Location	Details
Supabase Storage bucket <code>scans</code> (private)	Path <code>{userId}/{scanId}/{index}.jpg png</code> ; upload from the browser with upsert disabled (<code>lib/supabase/storage.ts:16-27</code>); reads only via 1-hour signed URLs minted server-side; deletes via <code>/api/scans</code> . Policies: SELECT/UPDATE/DELETE require <code>auth.uid() = owner</code> ; INSERT requires only “authenticated” — the path prefix is <i>not</i> checked, so any logged-in user can write into another user’s folder (they cannot read it back).
Supabase Storage bucket <code>avatars</code> (public)	Provisioned with policies but unused — avatars are stored as 150x150 base64 JPEG data-URLs in <code>profiles.avatar_url</code> (an explicit workaround for a Vercel header-size error, <code>profile/[slug]/page.tsx:118-123</code>). No DELETE policy exists on the bucket.
Browser sessionStorage	The full-resolution base64 capture (<code>wbh_scan_image</code>) and the analysis JSON persist for the browser session — cleared when the tab closes, not on logout.
Service-worker cache	<code>wbh-skin-v1</code> caches same-origin GET responses (pages, static assets); signed image URLs are cross-origin (<code>supabase.co</code>) and are not cached by it.
Public assets	<code>public/</code> holds marketing images, product photos (<code>public/prodctctspic/</code>), icons, manifest, <code>sw.js</code> . <code>assets/</code> at the repo root duplicates some of these for the legacy RN app.

21 Third-Party Services

Service	Purpose	Credentials	If it goes down
Supabase (hard dependency)	Auth (email + Google OAuth), PostgreSQL, Storage, RLS. SDKs: @supabase/ssr 0.10, supabase-js 2.101.	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY (public by design), SUPABASE_SERVICE_ROLE_KEY (server secret)	Login, profiles, history, bookings, products, admin — effectively the whole app — stop working.
Google Gemini (Generative Language API)	Vision skin analysis + text summary. Raw REST, no SDK.	GEMINI_API_KEY (server-side; passed in URL query string)	Scanning fails with the “high demand” 503 message; existing history still works; recommendations still work if given a stored analysis (summary falls back to a template).
Resend	All transactional email (booking x2, event registration x2, recommendation dispatch x2-3). Sender falls back to “WBH Skin <noreply@wbhskin.com>”.	RESEND_API_KEY; optional RESEND_FROM_EMAIL (not set in .env.local)	Bookings/registrations still save (email failures are logged, not fatal) but nobody is notified.
Google Fonts	Landing-page fonts via CSS link; app font via next/font.	—	Font fallbacks only.
Unsplash	Hardcoded fallback product image URL (recommend/route.ts:290) and legacy products.json images.	—	Broken fallback images.
wholesalebeautyhub.co.uk	The actual store — all purchase links, shop tab, product URLs.	—	Product links dead; no in-app impact.
WhatsApp (wa.me)	Landing FAB deep link — placeholder number 447000000000 .	—	—
Calendly UNUSED	Four env vars + one comment; zero functional code. Credentials should be revoked.	NEXT_PUBLIC_CALENDLY_URL, CALENDLY_CLIENT_ID/SECRET, CALENDLY_WEBHOOK_SIGNING_KEY, and ambiguous CLIENT_ID/CLIENT_SECRET	No impact.

Absent by design or omission: no payment processor, no SMS, no analytics, no error tracking / monitoring, no CDN configuration beyond the host’s defaults.

22 Environment Variables

Names only — values are never reproduced. `.env.local` is correctly gitignored and has never been committed (verified against git history). Note that only `.env*.local` is ignored — a plain `.env` would be committed if ever created.

Variable	Purpose	Client-exposed?	Required?
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL	Yes (intended)	Yes
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase public anon key (RLS-scoped)	Yes (intended)	Yes
SUPABASE_SERVICE_ROLE_KEY	Supabase service role — bypasses RLS	No ✓	Yes (admin + recommend + event flows)

GEMINI_API_KEY	Google Generative Language API	No ✓	Yes (scanning)
RESEND_API_KEY	Transactional email	No ✓	Yes for emails (soft-fails without)
RESEND_FROM_EMAIL	Sender identity; referenced in code, absent from .env.local (falls back to noreply@wbhskin.com)	No	Recommended
ADMIN_SECRET_PIN	Admin portal PIN — code contains a hardcoded fallback if unset	No	Must be set (and the fallback removed)
NEXT_PUBLIC_CALENDLY_URL, CALENDLY_CLIENT_ID, CALENDLY_CLIENT_SECRET, CALENDLY_WEBHOOK_SIGNING_KEY, CLIENT_ID, CLIENT_SECRET	Dead — zero references in src/. Revoke at provider and delete.	—	No

Positive finding: no secret carries a NEXT_PUBLIC_ prefix; the two public variables are exactly the two designed to be public. Negative: missing env vars for Supabase silently fall back to placeholder URLs (lib/supabase/client.ts:5-6, server.ts:8-9), turning misconfiguration into confusing runtime errors instead of failed builds.

23 Security Audit

Context that shapes everything below: `src/middleware.ts:18` excludes `/api/` from the auth middleware, so every API route must guard itself — and eight service-role routes do not. The admin UI’s PIN gate and sidebar role check are client-side only and are not security controls.

CRITICAL

Finding	Impact	Remediation
Unauthenticated PII dump: <code>GET /api/event-registration</code> (route.ts:287) returns every lead — name, email, phone, location, skin concerns — via the service-role client.	One anonymous GET exfiltrates the entire marketing lead list including health-related data. GDPR-reportable if exploited.	Add the admin guard; consider moving under <code>/api/admin/</code> .
Same data exposed a second way: the <code>event_registrations</code> <code>SELECT</code> policy is <code>USING (true)</code> (event_registrations.sql:27-28), so the public anon key shipped to every browser can read the whole table directly from the Supabase REST endpoint.	As above, independent of the API.	Drop the policy or restrict to <code>auth.role() = 'service_role'</code> .
Unauthenticated admin family: all of <code>/api/admin/products*</code> , <code>/api/admin/recommendations*</code> (GET/POST/PATCH + items), and <code>/api/admin/registration-recommendations</code> use <code>createAdminClient()</code> (RLS bypass) with no caller check .	Anonymous catalogue rewrite (PATCH <code>products/[id]</code>) applies the raw body — mass assignment, e.g. swap <code>product_url</code> to a malicious site); IDOR read of any user’s recommendations; triggering outbound branded email; arbitrary pre-confirmed account creation via <code>auth.admin.createUser</code> (registration-recommendations/route.ts:54).	Extract <code>requireAdmin</code> into <code>src/lib</code> and apply to every <code>/api/admin/*</code> handler; allowlist PATCH fields.
<code>profiles</code> <code>UPDATE</code> RLS policy lacks <code>WITH CHECK</code> (schema.sql:36-37).	Any registered user can PATCH their own row to <code>role='admin'</code> straight against Supabase REST with the anon key → full admin across the guarded endpoints too.	Add <code>WITH CHECK (auth.uid() = id)</code> and move <code>role/plan</code> to a separate admin-only table or column-level grant.
Unauthenticated, unmetered AI proxy: <code>POST /api/analyze</code> accepts up to 5 arbitrary files with no auth, size, or MIME checks and forwards them to a billed Gemini key.	Direct cost abuse (a script can burn the Gemini quota/budget) and use of your key as a free vision API.	Require a session, cap size/count/MIME server-side, and rate-limit.

HIGH

- Admin PIN theatre:** hardcoded fallback PIN in source (`verify-pin/route.ts:6`), non-constant-time compare, no rate limit/lockout; the “gate” is a `sessionStorage` flag settable in devtools; `(admin)/layout.tsx` has no server guard at all. Rotate the PIN (it is in git history), remove the fallback, and enforce roles server-side.
- Role escalation by admins:** any admin can promote anyone (including themselves) to admin via `PATCH /api/admin/users/[id]`; no superadmin requirement, no audit trail.
- HTML injection into email:** user-controlled names/notes are interpolated unescaped into every Resend template (`booking/route.ts:101,202`; `admin/recommendations/route.ts:278-404`) — combined with the unauthenticated PATCH, an attacker fully controls mail sent from the WBH domain to staff and customers.
- PostgREST filter injection:** search text is interpolated into `.or()` filters unescaped (`admin/users/route.ts:35`; `admin/products/route.ts:28-30` — the latter unauthenticated); sort column is unvalidated (`products/route.ts:50`).
- AI output trusted blindly:** no server-side validation before persisting to `scans.analysis` and rendering (`analyze/route.ts:216`).

MEDIUM

- Storage `scans` `INSERT` policy does not pin the path prefix to the uploader — cross-user writes possible (schema.sql:141-143). Same `INSERT` looseness on `avatars`.

- `DELETE /api/scans` with a malformed/empty body silently deletes **all** of the user’s scans (route.ts:91).
- Service worker caches authenticated pages into a shared browser cache with no logout invalidation — shared-device leakage risk (public/sw.js).
- Prompt injection into the Gemini summary from unvalidated client input (recommend/route.ts:127).
- No rate limiting anywhere (analyze, verify-pin, email-sending endpoints, public form). No CAPTCHA on the public form backed by a service-role insert.
- Uncapped/unvalidated `limit / page` `parseInt` on all list endpoints (?limit=1000000 is a free DoS).
- OAuth callback `next` parameter unvalidated (origin-prefixed, so internal-path redirection only).
- Password policy: 6 characters, client-side only.

LOW / INFORMATIONAL

- Gemini key in the URL query string rather than a header (leaks into URL-logging middleboxes).
- No security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) in next.config.ts.
- `requireAdmin` duplicated in four files with a hardcoded superadmin email in five — drift risk.
- PII (registrant emails) written to console logs (event-registration/route.ts:144,272).
- Dead credentials (Calendly, CLIENT_ID/SECRET) still provisioned; test_bucket.js scratch file in the repo root (harmless — anon key only).
- Positive: .env.local never committed; JWTs verified via `getUser()` rather than trusted from cookies; profile PATCH correctly allowlists fields; scans/booking/profile routes correctly session-guarded; scans bucket is private with signed URLs.

24 Privacy & User Data

Data	Where it lives	Who can access it	Deletion
Face photographs	Private Supabase Storage (scans bucket); transiently in browser sessionStorage as base64; transmitted to Google Gemini for analysis (Google API terms apply)	The owner (signed URLs); admins via the admin APIs; Supabase operators	User can delete per-scan or all scans (removes storage objects + rows). Not auto-expired.
AI analysis results (conditions, severities)	scans.analysis JSONB; sessionStorage cache	Owner + admins	Deleted with the scan
Identity & profile (name, email, phone, DOB field, age range, country, skin type, base64 avatar)	profiles table; email also in auth.users	Owner; admins; currently also readable cross-user if the RLS gaps in §23 are exploited	No user-facing account deletion exists
Event leads (name, email, phone, location, skin concerns)	event_registrations	Currently: anyone (two independent unauthenticated paths — §23)	No deletion path
Bookings (date, time, notes about skin concerns)	consultations; emailed to the aesthetician via Resend	Owner + admins + email recipients	User DELETE policy exists in SQL; no UI
Consent records	Nowhere — biometric consent is a UI checkbox only; cookie consent is a localStorage flag with no behavioural effect	—	—

Third-party transmission: images go to Google (Gemini); emails containing names, addresses and skin concerns go through Resend. **Compliance:** the PRD claims GDPR alignment, but this audit documents implementation only — the unauthenticated lead exposure, absent consent records, absent account deletion, absent retention policy, and the biometric-policy wording (“not permanently stored” while photos are stored) all require legal/product review before any compliance claim is made. UK GDPR treats biometric and health-adjacent data as special category — flagged, not assessed, here.

25 Error Handling

Scenario	What the user sees	What happens internally
Image too dark/bright or blurry (client gate)	"Image too dark or overly exposed — please retake..." / "Image is too blurry..."	Canvas validation rejects before any network call (imageValidation.ts:100-106)
No face / multiple faces / low quality (AI gate)	Friendly mapped message with retry	Gemini sets validation_error; no analysis performed (scan/page.tsx:284-296)
Gemini overloaded (429/503)	"Our AI is experiencing very high demand right now..."	3 retries with backoff per model, model fallback, then HTTP 503 (analyze/route.ts:67-114)
Gemini other error / empty / invalid JSON	"AI service error" / "AI returned an empty response" / "invalid response format"	502 with details only in server logs; markdown-fence salvage attempted for JSON
Analysis takes > 120 s	Timeout error state, retry offered	Client AbortController cancels the fetch
Scan save / image upload fails	Nothing — silent ; "Scan complete" still shows	bare catch {} and .catch(()=>{}) swallow it (scan/page.tsx:331-334)
Auth failure	401s surface as inline form errors; expired reset links map to friendly copy on /login	middleware redirects unauthenticated page loads to /login
Recommendation DB failure	"Database service error" / "No products configured"	502/500 from /api/recommend
Email failure	Nothing — booking/registration still succeeds	Resend errors are logged and swallowed by design
Rate limits	—	None exist

26 Performance Audit

- **Upload path sends full-size images** (up to 20 MB, +33% base64) — the camera path is fine (640x480 JPEG). Recommend: client-side downscale to ~1024 px before upload; Gemini needs no more.
- **Serial N+1 signed-URL minting** in four routes (e.g. scans/route.ts:66-69, admin/stats:101) — batchable with Promise.all or createSignedUrls.
- **Duplicate AI calls are possible**: nothing dedupes a re-submitted identical image; every /analysis confirmation re-POSTs /api/recommend (no caching of the product query, which loads the whole active catalogue each call).
- **O(n²) scan in the recommender** (diversified.includes inside a loop, recommendation-engine.ts:237) — trivial at ~310 products, worth fixing before the catalogue grows.
- **Admin stats pulls raw rows to aggregate in JS** (last 200 scans for top-conditions; every recommendation_item for product stats) — fine at current scale, should become SQL aggregates.
- **Base64 avatars in Postgres** inflate every profile read (including admin lists) by ~8–15 KB per user.
- **images.unoptimized: true** disables Next.js image optimisation — marketing images ship at full size.
- Positive: sessionStorage caches (scans 60 s, profile 5 min) cut repeat API chatter; static assets get 1-year immutable cache headers; skeletons keep perceived latency low.

27 Logging & Monitoring

- **Only console logging exists** (~30 console.log/error sites in route handlers). No Sentry, no analytics (GA/PostHog/Vercel Analytics), no uptime monitoring, no AI request/cost logging, no audit trail for admin actions.
- **PII reaches logs**: registrant email addresses are logged on successful sends (event-registration/route.ts:144, 272). Gemini error bodies are logged; image contents are not.
- Some routes swallow errors with bare catch {} (profile/route.ts:17,46) — those failures are invisible even in logs.
- Recommendation: add an error tracker, scrub PII from logs, and log per-request Gemini usage for cost attribution.

28 Deployment & Infrastructure

Layer	Finding
Frontend + backend hosting	Vercel is the evident target (inferred from a code comment about a Vercel 494 header error and the dev script's --max-http-header-size workaround) but no vercel.json, .vercel directory, Dockerfile, or CI config exists — deployment settings live only in the hosting dashboard. Build: <code>npm run build</code> (next build); serve: <code>npm start</code> . Note the header-size Node flag is applied in dev only.
Database / auth / storage	Supabase cloud project (URL in env). Provisioning = hand-running the SQL files; remember the three dashboard-only columns (\$18). Backups: not discoverable from the repo — Supabase plan-dependent.
AI	Google AI (Generative Language API) key; no project/region config in the repo.
Email	Resend account; the wbhskin.com sender domain must be verified in Resend for deliverability.
Auth email templates	emails/confirmation.html and emails/reset-password.html must be pasted into Supabase Dashboard → Auth → Email Templates (they use Supabase Go-template variables).
CI/CD, IaC, environments	None. No GitHub Actions, no tests to run, no staging convention in the repo. The "lint" script exists but no ESLint config does (it fails).
Domains	References to wbhskin.com (emails, hardcoded admin address) and wholesalebeautyhub.co.uk (shop). DNS/registrar details are outside the repo.

29 Project Structure

dermaAI_web/	
├─ src/	
│ └─ app/	
│ │ page.tsx	landing page (server component)
│ │ layout.tsx	root layout: fonts, PWA, ThemeProvider, CookieConsent
│ │ globals.css	1,661-line design system + dark theme
│ │ (auth)/	login, signup, forgot/reset-password, event-registration
│ │ (main)/	dashboard, scan, analysis, booking(+[id],success),
│ │ │ shop, call, profile(+[slug] with 7 sub-pages)	
│ │ (admin)/	admin portal (own root layout + admin.css)
│ │ │ └─ admin/	overview, users(+[id]), scans, products(+[id]),
│ │ │ │ activity, registrations, settings	
│ │ │ └─ components/	AdminSidebar, AdminTopbar
│ │ api/	18 route handlers (the entire backend)
│ │ auth/callback/	OAuth / email-link code exchange
│ └─ components/	AuthProvider, ThemeProvider, Navbar,
│ │ CompleteProfileModal, CookieConsent	
│ └─ landing/components/	7 landing-page sections
│ └─ lib/	
│ │ └─ supabase/	client / server / middleware / admin / storage
│ │ │ └─ recommendation-engine.ts	deterministic product matching
│ │ │ └─ imageValidation.ts	canvas quality gates
│ │ data/products.json	LEGACY static condition→product map (unused)
│ │ └─ middleware.ts	session refresh + page-level auth redirects
├─ supabase/	schema.sql, products_schema.sql (+v2 migration),
│ │ event_registrations.sql, promote_evelyn.sql,	
│ │ migrate_all_products.js, all_products.txt	
│ └─ supabase-setup.sql	OLDER divergent schema (reconcile & delete one)
├─ emails/	Supabase auth email templates + a misfiled homepage mockup
├─ public/	images, icons, manifest.json, sw.js, product photos
├─ _rn/	LEGACY Expo/React-Native scaffold (dead)
├─ assets/	LEGACY RN assets (dead)
├─ PRD.md	product requirements document (accurate, June 2026)
├─ README.md	STALE Expo boilerplate – does not describe this app
├─ test_bucket.js	scratch script (dead)
└─ next.config.ts · package.json · tsconfig.json	

30 Tech Stack Inventory

Technology	Purpose	Where	Importance
Next.js 15.3 / React 19 / TypeScript 5.9	Full-stack framework (App Router; route handlers are the backend)	entire src/	Core
Supabase (@supabase/ssr 0.10, supabase-js 2.101)	Auth, Postgres, Storage, RLS	src/lib/supabase, all API routes	Core
Google Gemini 2.5 Flash (+2.0 fallback)	AI vision analysis + summary text (raw REST)	api/analyze, api/recommend	Core
Resend	Transactional email (raw REST)	4 route files, 8 send sites	High
lucide-react 0.475	Icons	throughout	Low
recharts 3.8	Admin charts (bar, donut)	(admin) clients	Low
Vanilla CSS (globals.css + admin.css + inline)	Styling, light/dark theming	src/app	Core

Canvas / getUserMedia / WebAudio APIs	Capture, quality validation, avatar resize, ticket PNG export, shutter sounds	scan, imageValidation, profile, booking/success	High
PWA (manifest + hand-written service worker)	Installable app, offline-ish caching	public/	Medium
Expo / React Native	Legacy mobile scaffold — not in use	_rn/	Dead

31 Business Logic

Rule	Implementation
Closed 12-condition taxonomy	Prompt (analyze/route.ts:16-29) + AVOID_MAP + CONCERN_TAG_MAP must stay in sync manually
1 image per scan (UI), ≤20 MB, JPEG/PNG	scan/page.tsx:13,217,466 (API itself accepts up to 5 files of any type)
Quality gates	Luminance 30–230, sharpness >7.0 (client); face/single-face/quality (model)
Concern inclusion threshold	condition confidence ≥ 30 (recommendation-engine.ts:271)
Primary vs secondary concerns	severity Severe/Moderate → primary; else secondary (recommend/route.ts:163-169)
Skin-type augmentation	oily/combination adds “Oily Skin / Excess Sebum”; dry adds “Dehydration” (recommend/route.ts:153-158)
Product scoring	tagOverlap×0.40 + coverageRatio×0.30 + confidenceWeight×0.30 (High 1.0 / Medium 0.7 / Low 0.4); max 3 per category; ≥1 from each routine category; top 6 returned
Routine assembly	Cleanse → Tone → Target (≤2 treatments; acids/retinoids evening-only by keyword) → Moisturise → SPF (morning) with sensible defaults when a category is missing (recommend/route.ts:179-235)
Fabricated prices	getProductPrice() hashes the product name into £12–£45 with .49/.99/.00 endings (recommend/route.ts:52-60). BUSINESS RISK — displayed beside real purchase links; likely to mismatch actual store prices. Same for “key ingredient” labels (keyword guessing, route.ts:62-92).
Skin score	Stored: 100 – mean condition confidence; displayed: the AI’s image-quality confidence_score — inconsistent semantics (scan/page.tsx:312; analysis/page.tsx:123)
Booking	Single hardcoded specialist (£50 shown, never charged); 5 fixed slots; dedupe on user+doctor+date+time; “Fee Status: Paid” hardcoded in the email
Scan/usage limits	None — unlimited free scans per user; plan free/pro changes nothing
Affiliate/UTM links	Missing product URLs fall back to wholesalebeautyhub.co.uk/product/{slug}/?utm_source=wbhskin (recommend/route.ts:295)

32 AI Cost & Usage Considerations

- Calls per completed scan:** 1 vision call (up to 3 retries per model × 2 models on overload — retries are re-billed only when they succeed, but each successful attempt is billed once) + 1 small text call when recommendations are requested. Worst-case latency budget before the client aborts: 120 s.
- Payload driver:** the uploaded image at original size, base64-inflated ~33%. The camera path (~640×480 JPEG) costs roughly an order of magnitude less input than a full-resolution 15–20 MB upload. Client-side downscaling is the single biggest cost lever available.
- Output budget:** vision maxOutputTokens 8192 (typical responses are far smaller); summary capped at 200 tokens.
- No caching or dedupe:** identical images are re-analysed in full; the recommendation summary is regenerated per confirmation click.
- Cost exposure:** because /api/analyze is unauthenticated and unmetered, current worst-case spend is set by attackers, not users (Section 23). Google may also apply free-tier rate limits that legitimate traffic would hit unpredictably.

- **Pricing is not in the repository** and changes frequently — verify current Gemini 2.5 Flash pricing and quota with Google AI Studio / Cloud billing before making cost projections.

33 Limitations of the Current Application

Product

- No payments (booking fee and “Paid” status are cosmetic; plan free/pro does nothing).
- Displayed product prices are fabricated (§31).
- Single hardcoded specialist and 5 static time slots — no real availability calendar.
- No account deletion, working notifications, 2FA, or language switching (buttons exist, do nothing).
- /call is a mock; “Report” download button is dead; Scan Diagnostics values are hardcoded.
- WhatsApp number and support emails are placeholders.

Technical

- No tests, no CI/CD, no error tracking, no analytics; stale README; two divergent schema files plus dashboard-only columns.
- Backend logic is inlined per-route (auth guard copy-pasted; ~800 lines of duplicated email HTML).
- Dead weight: _rn/, assets/, src/data/products.json, Calendly vars, test_bucket.js.

AI

- Uncalibrated self-reported confidence; no validated accuracy; closed 12-condition cosmetic list (anything else is invisible); prompt-only schema enforcement; no safety settings; no response validation.

Security & privacy

- The critical/high findings in §23; consent not recorded; no retention policy; no rate limiting.

Performance & scalability

- Full-size upload path; N+1 signed URLs; in-JS aggregation for admin stats; base64 avatars in Postgres; single-region serverless is otherwise fine for expected scale.

UX

- Dark mode partial (inline styles and admin ignore it); login redirect loses the intended destination; silent scan-save failures; HEIC uploads fail confusingly.

34 What the Client Owns / Depends On

Asset	What to secure at handover
Source repository	github.com/OSAMOR19/dermaAI_web — transfer or fork to a client-owned org; note the repo currently lives under a personal account.
Hosting account	The Vercel (or equivalent) project holding the production deployment, domains, and all environment variables — the env values exist nowhere else.
Supabase project	Owns all user data, auth, storage, RLS config, the three dashboard-only columns, and the auth email templates. Transfer org ownership; export a full schema dump to close the drift (§18).
Google AI key	GEMINI_API_KEY and its billing account/quota. Rotate on handover.
Resend account	API key + verified sending domain (wbhskin.com). Rotate on handover.
Domains	wbhskin.com (app + email identity) and wholesalebeautyhub.co.uk (store, external).
Admin PIN + role grants	ADMIN_SECRET_PIN env var; profiles.role assignments (promote via SQL or the admin UI); the hardcoded superadmin email in five source files.

Dormant credentials	Calendly + CLIENT_ID/SECRET — revoke at the providers.
Monitoring/analytics	None exist to hand over.

35 Maintenance Guide

Run locally (commands verified from package.json)

```
git clone https://github.com/OSAMOR19/dermaAI_web && cd dermaAI_web
npm install
# create .env.local with: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY,
# SUPABASE_SERVICE_ROLE_KEY, GEMINI_API_KEY, RESEND_API_KEY, RESEND_FROM_EMAIL,
# ADMIN_SECRET_PIN (values from the hosting dashboard / password manager)
npm run dev      # Next.js dev server on :3000 (Turbopack)
npm run build && npm start  # production build + serve
```

There is no seed script for a fresh database: run supabase/schema.sql, supabase/products_schema.sql, supabase/products_migration_v2.sql, supabase/event_registrations.sql in the Supabase SQL editor, then ALTER TABLE profiles ADD COLUMN role/plan/location (§18), then optionally node supabase/migrate_all_products.js for the full catalogue. Paste emails/confirmation.html and emails/reset-password.html into Supabase auth templates. “npm run lint” fails — no ESLint config exists.

Common changes

- **Modify the AI prompt:** edit DERMA_PROMPT in src/app/api/analyze/route.ts (and keep §36 in mind).
- **Modify recommendations:** src/lib/recommendation-engine.ts (mapping/weights) and /api/recommend (routine assembly, avoid list, pricing display).
- **Add products/categories:** Supabase products/categories tables (dashboard or migration script); tags drive matching.
- **Change the specialist or slots:** hardcoded in (main)/booking/page.tsx, booking/[id]/page.tsx, and DOCTOR_EMAILS in /api/booking.
- **UI:** pages under src/app; shared styles in globals.css / admin.css (expect heavy inline styles).
- **Deploy:** push to main → hosting auto-build (dashboard-configured); no CI gates exist.

36 How to Add a New Skin Condition

- The taxonomy is duplicated across independent constants — all must change together (there is no single source of truth):
1. **Prompt list** — add the exact display string to the 12-condition list in DERMA_PROMPT (api/analyze/route.ts:16-29).
 2. **Avoid list** — add a lowercase key with “avoid” items to AVOID_MAP (api/recommend/route.ts:21-34).
 3. **Tag mapping** — add the lowercase key → product-tag array in CONCERN_TAG_MAP (src/lib/recommendation-engine.ts:38-84).
 4. **Products** — ensure at least one active product carries a matching tag in products.skin_concern_tags, or the condition will recommend nothing.
 5. **Optional:** per-condition treatment copy in the legacy src/data/products.json (unused), and any admin filter lists that enumerate conditions.
 6. **No database or UI schema change is needed** — conditions are free strings in JSONB; cards render generically.
 7. **Test manually** (no test suite): run a scan of a matching image, confirm the condition appears, is confirmable, and yields products.

37 How to Change the AI Model

The integration is a raw fetch in exactly two files — swapping models within Gemini is a constant change; swapping providers means rewriting the request/response mapping in both.

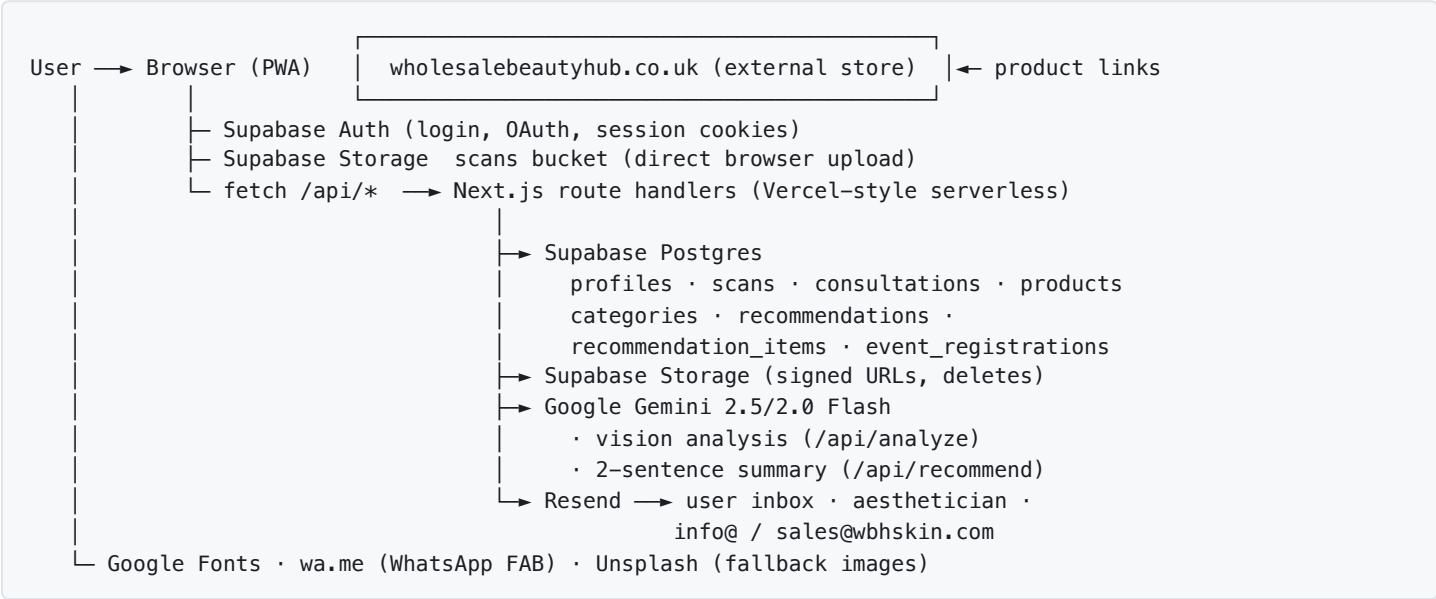
1. **Within Gemini:** edit GEMINI_MODELS (analyze/route.ts:58-61) and the summary URL constant (recommend/route.ts:6). Done.
2. **To another provider:** replace the endpoint + auth header; convert the parts array ({text} + {inline_data:{mime_type,data}}) to the provider's image format; keep DERMA_PROMPT (it is provider-agnostic); map the response-text extraction (currently candidates[0].content.parts[0].text); preserve the retry/backoff wrapper and the JSON-parse + fence-salvage logic; swap GEMINI_API_KEY for the new env var; update the retryable status codes.
3. **Checklist:** ☐ new env var set in hosting ☐ old key revoked ☐ validation_error behaviour re-tested (face/multi-face/quality) ☐ JSON adherence spot-checked (add a real response schema if the provider supports one) ☐ latency within the 120 s client timeout ☐ cost re-estimated ☐ the admin settings display string updated (SettingsClient.tsx:152 hardcodes "Gemini 2.5 Flash").

38 Testing Audit

There are no tests. No unit/integration/e2e tests, no test framework in package.json, no CI to run them. The only verification artifact is test_bucket.js, an 8-line manual scratch script with no assertions. Recommended minimum test checklist, in priority order:

1. API auth matrix: every /api/admin/* and /api/analyze rejects anonymous callers (this doubles as the fix verification for §23).
2. /api/analyze: mocked Gemini — happy path, each validation_error, invalid JSON, 503 retry/fallback path.
3. recommendation-engine unit tests: tag expansion, scoring weights, category diversification, ≥ 30 threshold.
4. /api/recommend: routine assembly rules (SPF morning-only, acids evening-only), fabricated-price function (before/after replacing it).
5. RLS tests against a scratch Supabase project: profiles self-update cannot change role; event_registrations not readable with the anon key.
6. E2E (Playwright): signup → scan (fixture image, mocked AI) → analysis → recommendation → booking.
7. DELETE /api/scans body-parsing edge case (empty body must not mean "delete all").

39 Complete Dependency Map



40 Source-Code Evidence Convention

Every material claim in this document carries an inline `file:line` reference to the audited tree (repository root = `dermaAI_web`). Example of the convention used throughout:

File: `src/app/api/analyze/route.ts`
Function: `POST() + callGeminiWithRetry()`
Purpose: Accepts multipart images, base64-encodes them, calls Gemini 2.5-flash (fallback 2.0-flash) with `DERMA_PROMPT`, parses the strict-JSON response, returns it verbatim to the client.

The most load-bearing files for a new developer, in reading order: `src/app/api/analyze/route.ts` → `src/app/(main)/scan/page.tsx` → `src/app/(main)/analysis/page.tsx` → `src/app/api/recommend/route.ts` → `src/lib/recommendation-engine.ts` → `supabase/schema.sql` → `src/lib/supabase/middleware.ts`.

41 Unknown / Unverified Information

Item	Status	Reason / what is needed
AI clinical accuracy	NOT FOUND	No validation study or figure exists in the code. Requires a dermatologist-labelled evaluation before any accuracy claim.
Live production infrastructure (hosting project, domains, DNS, backups)	INFERRED	Vercel inferred from code comments only; no deployment config in the repo. Client must confirm hosting account details.
Live database state (role/plan/location columns, whether RLS matches the SQL files, applied policy set)	PARTIALLY CONFIRMED	Code proves the columns exist in production; the SQL files do not create them. Export the live schema to close the gap. Whether the profiles-RLS escalation gap is exploitable as written depends on the live policy set.
Which schema file was actually applied (supabase-setup.sql vs supabase/schema.sql)	UNKNOWN	Two divergent files; reconcile against a live dump.
Gemini & Resend pricing, quotas, contracts	REQUIRES CLIENT CONFIRMATION	Not in the repo; check provider dashboards/billing.

GDPR / UK GDPR / medical-device compliance posture	REQUIRES LEGAL REVIEW	This audit documents the implementation (§24) — several observed gaps (lead exposure, no consent records, no deletion/retention) make current compliance claims unsafe.
Data retention obligations & Supabase backup policy	NOT FOUND	Nothing in code auto-deletes; backups are plan-dependent.
Ownership of the info@wbhskin.com superadmin mailbox & Evelyn’s admin grant	REQUIRES CLIENT CONFIRMATION	Hardcoded email is a standing backdoor; confirm who controls it.
Purpose of CLIENT_ID / CLIENT_SECRET env vars	UNKNOWN	Unlabelled, unused; identify and revoke.
Real product prices	CONFIRMED FABRICATED IN CODE	Displayed prices are name-hash outputs (§31); syncing real prices needs a WooCommerce feed or manual price column.

42 Final Client-Friendly Explanation

What it does. WBH Skin is your AI skincare adviser website. A customer signs up, agrees to the consent notice, and takes or uploads a selfie. Google’s Gemini AI looks at the photo and reports which of 12 common skin concerns it can see, how confident it is, and how severe each looks. The app turns that into a skin score, a morning-and-evening routine, and a shortlist of products from your shop, with links to buy. Customers can save their scan history, and can book a session with your aesthetician — both of you get a confirmation email. Your team has a private dashboard to see users, scans and event leads, manage the product catalogue, and email curated product picks to customers.

Where the data goes. Photos and results are stored in your Supabase cloud database (photos in a private bucket only the owner can view); each photo is also sent once to Google for the AI to analyse; emails go out through Resend. Nothing is sold or shared beyond those services.

What it runs on. One codebase (Next.js/React) that serves both the website and its API, your Supabase project (accounts, database, image storage), Google Gemini (the AI), and Resend (email). No payment system is connected — purchases happen on your existing WooCommerce store, and the consultation “fee” is display-only.

What needs attention first. (1) A handful of admin API doors were left unlocked — including the event-lead list, which anyone on the internet could currently download, and the product catalogue, which anyone could edit. These are quick fixes for a developer but they matter for data-protection law, so they should be done before any marketing push. (2) The product prices shown in recommendations are made-up numbers, not your shop prices. (3) The AI’s results are informational, not medical — keep the disclaimers, and don’t quote an accuracy figure until one is properly measured. (4) There are no automated tests, so changes should be made carefully and checked by hand.

What a future developer needs. Access to the GitHub repo, the hosting account (with the environment variables), the Supabase project, the Google AI key, and the Resend account. With those, Sections 35–37 of this document tell them how to run, change, and redeploy everything.

43 Final Technical Summary

Frontend	Next.js 15.3 (App Router) · React 19 · TypeScript 5.9 · vanilla CSS · PWA (manifest + custom service worker) · Recharts (admin) · lucide-react
Backend	Next.js route handlers (serverless, Node runtime) — no separate server
Database	Supabase PostgreSQL, RLS, 1 trigger/function (handle_new_user); hand-run SQL files, no migration tool
Authentication	Supabase Auth: email/password + Google OAuth; cookie sessions via @supabase/ssr; roles user/admin/superadmin in profiles.role
AI	Google Gemini — gemini-2.5-flash (vision + summary) with gemini-2.0-flash fallback; raw REST; temp 0.2/0.4; JSON-by-prompt
Image processing	Browser canvas: capture 640×480 JPEG 85%, luminance/sharpness gates, 150×150 avatar resize; server: base64 encode only

Storage	Supabase Storage: scans (private, signed URLs 1 h), avatars (public, unused)
Email	Resend REST — booking, event registration, recommendation dispatch (8 send sites); Supabase-hosted auth emails
Deployment	Vercel-style host (inferred); no CI/CD, no IaC, no containers
Monitoring / analytics / tests	None / none / none
Counts	31 screens (16 user/public pages + 7 profile sub-screens + 9 admin, incl. 1 mock) · 18 API route files / 28 endpoints · 8 database tables + 2 storage buckets + 1 trigger · 3 active external services (Supabase, Google Gemini, Resend) + 1 dormant (Calendly) · 2 AI integrations (vision analysis, text summary) · 12 detectable skin conditions · ~310 catalogued products in 18 categories

44 Final Audit Checklist

A second full-repository sweep was performed for every term in the brief’s checklist (fetch(, axios, /api/, supabase, firebase, auth, storage, upload, download, AI, Gemini, OpenAI, Vertex, Anthropic, model, prompt, image, camera, file, database, query, mutation, HTTP verbs, webhook, token, session, process.env, NEXT_PUBLIC_). Outcomes:

- **axios, Firebase, OpenAI, Vertex, Anthropic, GraphQL, webhooks, server actions:** zero occurrences — confirmed absent.
- **fetch(/ /api/:** all ~30 internal call sites and all 10 external call sites are accounted for in Sections 17 and 21.
- **process.env / NEXT_PUBLIC_:** all 23 reference sites reconciled against .env.local names (Section 22); no secrets client-exposed; six dead variables flagged.
- **prompt / model / Gemini:** both prompts reproduced verbatim (Section 11); the only other “model” hits are the hardcoded admin display string and dead constants.
- **upload / storage / camera / image:** the full pipeline is traced end-to-end in Sections 10 and 20, including the unused avatars bucket and the base64-avatar workaround.
- **token / session:** Supabase cookie sessions only; no custom JWT handling; sessionStorage keys enumerated in Section 4.
- Second-pass additions folded into this document: the dead single-file “file” field in /api/analyze, the unused GEMINI_URL constant, the dns.setDefaultResultOrder('ipv4first') calls in booking/event routes, the misfiled homepage mockup in emails/, and the .gitignore gap for a plain .env.